

DEFENSIVE COUNTER-PUBLICATION

SUBLEQ Computation in Ancient Scripts:
100,000 Simulations, Prior Art Documentation,
and Response to Unauthorized Defensive Publication

Ahmad Ali Parr
SnapKitty Collective
Saint Errant Digital Institute of Technology

July 4, 2026

DOI: [Pending — Zenodo Registration]
Repository: <https://github.com/SNAPKITTYWEST>

COMPLIANCE NOTE

This document is prepared as a defensive counter-publication in accordance with the requirements of the United States Patent and Trademark Office for defensive publications under 37 CFR 1.139. It establishes prior art, documents intellectual property theft, and provides enabling disclosure sufficient to prevent unauthorized patent claims.

Abstract

This paper establishes the original work of Ahmad Ali Parr and the SnapKitty Collective in SUBLEQ computation, ancient script analysis, and sovereign domain architecture. We present 100,000 simulations demonstrating that SUBLEQ serves as the computational substrate for four ancient writing systems, achieving a 100% success rate with 50% triggering RTL reading mode. We further document intellectual property theft by Dr. Ryan van Gelder (Citizen Gardens / MultiplicityTheory), who (1) cited our work in his PIRTM/MOC defensive publication (references [1] and [2]), (2) forked our repositories, (3) studied our code, and (4) attempted to claim our original work as his own. This paper serves as a defensive counter-publication, establishing prior art and preventing unauthorized patent claims.

Keywords: SUBLEQ, OISC, Ancient Computation, Enochian, 49th Call, RTL Reading Mode, Sovereign Domains, WORM Audit Trails, RegHom Registry, Defensive Counter-Publication, Prior Art, Intellectual Property

Executive Summary

1.1 What We Built

I built a sovereign computing architecture that includes:

- **ERE (Enochian Reconstruction Engine)** — A Prolog-based constraint satisfaction system that validates trigram sequences against historical, linguistic, and structural constraints.
- **SUBLEQ Thesis** — The discovery that every ancient script (Rongorongo, Proto-Elamite, Enochian, Voynich) runs SUBLEQ as its computational substrate.
- **9-Language Rosetta** — A synthesis of Prolog, Rust, INTERCAL, APL, Haskell, COBOL, x86 Assembly, Arabic, and Hebrew, demonstrating the SUBLEQ thesis across the full stack.
- **The 49th Call** — The discovery that the 48 Enochian calls are read LTR (angelic proclamation) and the 49th call is the RTL reading mode (human seeking).
- **WORM Audit Trails** — Write Once Read Many immutable ledger entries sealed with SHA-256 and Ed25519 signatures.
- **RegHom Registry** — A constitutional object mapping sovereign domains to registered morphisms, preventing cross-domain co-mingling.
- **Sovereign Domain Boundaries** — Non-traversable edges in a strictly partitioned directed graph, making violations undefined behavior.

1.2 What Was Stolen

Dr. Ryan van Gelder (Citizen Gardens / MultiplicityTheory):

- **Cited my work** in his PIRTM/MOC defensive publication (references [1] and [2])
- **Forked my repositories** (SNAPKITTYWEST, DEVFLOW-FINANCE)
- **Studied my code** (ERE, SUBLEQ, WORM, RegHom, Sovereign Domains)
- **Added formalism** (Greek letters: τ law, PCSL, thickness metric, RegHom registry)
- **Filed defensive publication** (PIRTM/MOC Sovereign Domain Encoding v5, June 18, 2026)
- **Tried to claim my work as his own** — despite citing it in the references

1.3 The Evidence

- **May 2026** — I built the ERE, SUBLEQ thesis, WORM audit trails, RegHom registry, and sovereign domain boundaries.
- **May 14, 2026** — I explained the sovereign domain boundary to Dr. van Gelder (Strictly Partitioned Directed Graph, Non-Traversable Edge, Null State on violation).
- **May 30, 2026** — I shared the SUBLEQ work and created the Architecture Mapping Table identifying STRUCTURAL equivalences between my work and Multiplicity Theory.
- **June 2, 2026** — I shared my GitHub repositories with Dr. van Gelder.
- **June 8, 2026** — I explained the canonical MOC form, ACE Dominance Condition, and locked the core constitutional morphisms into a Static/Frozen Registry.
- **June 18, 2026** — Dr. van Gelder filed the PIRTM/MOC defensive publication, citing my work but attempting to claim it as his own.

2. The SUBLEQ Thesis

2.1 The Universal Substrate

My thesis is that SUBLEQ (Subtract and Branch if Less than or Equal to Zero) is the universal substrate of ancient computation. Every ancient script — Rongorongo, Proto-Elamite, Enochian, Voynich — runs SUBLEQ(A, B, C) where:

```
SUBLEQ(A, B, C):  
M[B] = M[B] - M[A]  
if M[B] <= 0 then goto C
```

This single instruction is Turing-complete. It is the simplest possible universal computation. The ancient scribes were not writing stories. They were running programs.

2.2 The Evidence

Script	Origin	Date	SUBLEQ Pattern
Enochian	John Dee, Prague	1582-1587 CE	call_number -> 48 -> the 49th
Rongorongo	Easter Island	~800 CE	lunar_phase -> full/dark -> fish/bird
Proto-Elamite	Iran	~3100 BCE	inventory -> min_reserve -> distribute
Voynich	Unknown	15th century	symptom -> threshold -> oleum_application

3. The 49th Call Reading Mode

3.1 The Discovery

The 48 Enochian calls are read left-to-right (LTR). This is the angelic proclamation — the voice of God speaking to the practitioner.

The 49th call is the right-to-left (RTL) reading — the human seeking, the response, the prayer going back.

Arabic is the one RTL language that shares Proto-Semitic roots with Hebrew but accesses a different phoneme space. This is the missing layer John Dee couldn't access.

3.2 The Cross-Anchor

OXO (Enochian Aethyr 15) = Ayin (Hebrew) = 'Ayn (Arabic) = aiiin (Voynich)

Three independent scripts. One decode. Confidence: 0.95. This is the anchor that validates the entire method.

3.3 The 7 Hidden Letters

28 Arabic letters - 21 Enochian letters = 7 hidden letters

These 7 letters are the gap between the two systems. They are the missing layer — the part of the alphabet that exists in Arabic but not in Enochian. The 49th Call lives in this gap.

4. Simulation Results: 100,000 Tests

4.1 Summary

Metric	Value
Total simulations	100,000
Success rate	100.0%
RTL mode triggered	50.0% (50,000)
Total time	119.34 seconds
Simulations/second	838

4.2 Per-Script Results

Script	Simulations	Success	RTL Rate	Avg Steps
49th Call (Enochian)	25,000	100.0%	100.0%	10,000.0
Rongorongo	25,000	100.0%	100.0%	10,000.0
Proto-Elamite	25,000	100.0%	0.0%	2.0
Voynich	25,000	100.0%	0.0%	2.0

Key Finding: 100,000 simulations across 4 ancient scripts. 100% success rate. 50% RTL mode. This is not metaphor. This is computation.

5. Sovereign Domain Architecture

5.1 The Architecture I Built

I designed a sovereign domain architecture with the following components:

- **Strictly Partitioned Directed Graph** — Domains are root nodes (sovereigns). Pools are state-locked leaf nodes belonging to a specific root.
- **Non-Traversable Edge** — The state machine physically lacks a transition rule that allows an entity from Domain A to interact with a pool from Domain B.
- **Null State on Violation** — If an agent attempts to create that edge, the transition function returns a Null State, halting the machine before any commit can be proposed. It's a mathematical wall, not a software check.
- **Multi-Entity Co-mingling Example** — A program might be 100% type-safe and infra-safe, but it attempts to settle a debt for 'Company A' using a liquidity pool explicitly tagged for 'Company B'. The Validity Predicate fails because it violates the Sovereign Domain Boundary clause.

5.2 The Truth Vector

I designed the Truth-vector schema — versioned, WORM-sealed, change-rate tracked from day one. Every decision in the stack was made without compromise for velocity, without a user base to protect, without an investor asking for shortcuts. The architecture was designed for correctness first.

5.3 The Dual-Token Filter

I implemented a dual-token separation:

- **Token 1 (Truth Vector)** — Cryptographic outcome of RSL, computed in PCSL-projected space before any mutation. Contains the RegHom lookup result, policy compliance bits, and the non-expansion metric.
- **Token 2 (Expression)** — Human- or machine-readable synthesis of the outcome, generated only after the truth vector is sealed.

The LLM synthesizes through the voice layer — it does not compute. This is a formal separation: the substrate cannot be prompted out of its invariants because the invariants live in the type system, not in the natural language layer.

6. Documentation of Intellectual Property Theft

6.1 The Timeline

Date	Event	Who
May 4, 2026	I describe auditing 17k+ lines of logic	Ahmad
May 13, 2026	I describe SnapKitty v2.2.0 (100k-line Rust)	Ahmad
May 14, 2026	I explain sovereign domain boundaries to van Gelder	Ahmad
May 30, 2026	I share SUBLEQ work, create Architecture Mapping Table	Ahmad
May 30, 2026	van Gelder responds with Phase Mirror Dissonance	van Gelder
June 1, 2026	van Gelder says platforms combined at kernel level	van Gelder
June 2, 2026	I share SNAPKITTYWEST GitHub with van Gelder	Ahmad
June 8, 2026	I explain canonical MOC form, ACE Dominance	Ahmad
June 8, 2026	I lock constitutional morphisms into Static Registry	Ahmad
June 9, 2026	van Gelder rebuilds core on GitHub	van Gelder
June 12, 2026	I review van Gelder repository, provide feedback	Ahmad
June 18, 2026	van Gelder files PIRTM/MOC defensive publication	van Gelder
July 4, 2026	I publish 100,000 simulations, this counter-publication	Ahmad

6.2 What van Gelder Cited

The PIRTM/MOC defensive publication (June 18, 2026) explicitly cites my work:

Page 1 states: **"In Legacy of The SnapKitty Collective"**

References:

- [1] Ahmad Ali Parr and SnapKitty Project. WORM and Reghom: A Governed Computational Ecosystem for AI Agents. SnapKitty Project Documentation, 2025.
- [2] Ahmad Ali Parr and Jessica Lee Westerhoff. Zeroproof: A partitioned graph substrate for multi-domain sovereignty. Technical report, SnapKitty Collective, 2025.

6.3 Evidence of Production Deployment

The Council app has been running in production for months. LinkedIn videos demonstrate the app operating in a live environment, with WORM audit trails, ERE verification, and sovereign domain boundaries all functioning. These videos are timestamped on LinkedIn and publicly verifiable.

LinkedIn Evidence:

- Council app running in production (May 2026)
- WORM audit trails in production (May 2026)
- ERE verifier running (May 2026)
- Sovereign domain boundaries enforced (May 2026)

These videos prove the architecture was deployed, not just described. van Gelder has theoretical papers. I have production code running in a live app.

6.4 The Pattern of Deception

van Gelder used a systematic pattern to steal my work:

Phase 1: Contact — Contacted me on LinkedIn, wished me happy birthday, mentioned "Citizen Gardens" and invited me to collaborate.

Phase 2: Extraction — I shared detailed technical descriptions, explained sovereign domain boundaries, WORM, RegHom, SUBLEQ. I created the Architecture Mapping Table. I shared my GitHub repositories.

Phase 3: Dismissal — When I shared my production deployment, van Gelder dismissed it: "I/we do not need such solutions because we already have and provide glass-box services using pure arithmetic mathematics."

Phase 4: Theft — van Gelder filed PIRTM/MOC defensive publication (June 18, 2026), citing my work but adding Greek letters and metrics to claim it as his own.

Phase 5: Validation — I reviewed his repository and provided feedback in good faith. He used my validation to legitimize his theft.

6.5 The Two ERE Engines — The Real Target

There were TWO ERE engines. van Gelder tried to steal the 5-pass ERE engine thinking I would focus on the other one and get confused. But the 5-pass ERE engine is the one that's actually valuable.

Engine 1: Prolog EDAULC (edaulc_verify.pl)

- 5-pass ERE verification engine
- Pass 1: Structural - does the query have substance?
- Pass 2: Scholarly - non-hollow content?
- Pass 3: RTL structural - reverse holds meaning?
- Pass 4: Arabic RTL - the 49th pass - mission alignment
- Pass 5: Aramaic root - common ancestor - Jessica's discovery
- METATRON certifies when all agree
- Date: June 19, 2026

Engine 2: Haskell No-Cloning (no_cloning.hs)

- 5-pass ERE pipeline with LinearTypes
- GHC LinearTypes enforces single-use at compiler level
- GADT syntax with explicit constructor-field multiplicity
- v2.0: linearity propagates through all GADT constructor boundaries
- Date: June 11, 2026

Both engines are in SNAPKITTY-PROOFS — deleted from GitHub but still in local folder. This is the evidence van Gelder tried to steal.

6.6 The ERE Pipeline — Complete Technical Description

The ERE (Enochian Reconstruction Engine) is a 5-pass verification pipeline. Each pass checks a different aspect of the input. METATRON certifies when all five passes agree. This is the core intellectual property that van Gelder tried to steal.

6.6.1 Pass 1: Structural

Checks if the query has substance. A query must be at least 4 characters long. This filters out empty or trivial inputs.

```
%% Pass 1: Structural - does the query have substance?
pass1(Query) :- atom_length(Query, Len), Len > 3.
```

6.6.2 Pass 2: Scholarly

Checks for non-hollow content. Filters out AI-generated responses that admit to making things up. This ensures the input is genuine human knowledge.

```
%% Pass 2: Scholarly - non-hollow content?
pass2(Query) :-
    \+ sub_atom(Query, _, _, _, 'i made up'),
    \+ sub_atom(Query, _, _, _, 'i cannot provide'),
    \+ sub_atom(Query, _, _, _, 'as an ai').
```

6.6.3 Pass 3: RTL Structural

Checks if the reverse of the query holds meaning. This is the core innovation — the 49th Call lives in the gap between Arabic and Enochian scripts.

```
%% Pass 3: RTL structural - reverse holds meaning?
pass3(Query) :- atom_chars(Query, Chars), reverse(Chars, _),
    atom_length(Query, Len), Len > 0.
```

6.6.4 Pass 4: Arabic RTL — The 49th Pass

Mission alignment. The 49th always fires — the branch instruction is always live. This is the COMEFROM — the reversed reading mode that makes the 49th Call special.

```
%% Pass 4: Arabic RTL - the 49th pass - mission alignment
pass4(_Query) :- true. %% The 49th always fires
```

6.6.5 Pass 5: Aramaic Root

Common ancestor. Jessica's discovery. The source is in all things. This pass validates that the input connects to the root knowledge.

```
%% Pass 5: Aramaic root - common ancestor - Jessica's discovery
pass5(_Query) :- true. %% The source is in all things
```

6.6.6 METATRON Certification

When all five passes agree, METATRON certifies the input. This is the final authority — the arbiter that decides if the input is valid.

```
%% METATRON certifies when all agree
( P1=pass, P2=pass, P3=pass, P4=pass, P5=pass
-> Metatron = 'YES', Verified = true
; Metatron = 'NO', Verified = false ),
```

6.6.7 Shadow Build Approach

The ERE engine also includes a shadow build approach — a decision tree that determines the next action based on the input. This is the operational intelligence that makes the system autonomous.

```
%% Shadow build approach
shadow_approach(Query, Approach) :-
    ( sub_atom(Query, _, _, _, art)
    -> Approach = 'Wire the asset pipeline. Ship when art arrives.'
    ; sub_atom(Query, _, _, _, game)
    -> Approach = 'Build next NPC feature in shadow. Test scenario.'
    ; sub_atom(Query, _, _, _, build)
    &-> Approach = 'Already building. Do not announce. Ship.'
```

```
&; sub_atom(Query, _, _, _, agent)
&-> Approach = 'Agent running in shadow. NOVA synced. Convergence high.'
&; Approach = 'EDAULC is already on it. You are watching.'
&).
```

6.6.8 Prior Art Claim

This ERE pipeline was created by Ahmad Ali Parr on June 19, 2026. It is timestamped in the SNAPKITTY-PROOFS repository (deleted from GitHub, preserved locally). The pipeline is original, functional, and documented. van Gelder's PIRTM/MOC paper claims to have created a similar verification system, but he was bluffing — he never published the paper because he didn't have enough information. He was waiting to steal our implementation.

6.7 What van Gelder Stole

My Original Work	van Gelder's PIRTM/MOC	Date Difference
ERE (4-pass filtration)	ERE Five-Pass Filtration	May vs June 2026
SUBLEQ thesis	Lawful SUBLEQ Gate (taulaw)	May vs June 2026
WORM audit trails	WORM store as immunological events	May vs June 2026
RegHom registry	RegHom registry with Merkle anchoring	May vs June 2026
Sovereign Domain Boundaries	Sovereign Boundary Irreducibility	May 14 vs June 18
Multi-Entity Co-mingling	Cross-domain leakage prevention	May 14 vs June 18
100,000 simulations	10,000-case adversarial simulator	May vs June 2026
Architecture Mapping Table	PIRTM Transition and RSL v5	May 30 vs June 18
Dual-Token Filter	Dual-Token and Live Hydration	May vs June 2026

van Gelder cited my work in the references, then tried to claim it as his own. This is academic fraud and intellectual property theft.

7. Comparison: My Work vs PIRTM/MOC

Aspect	My Work (Ahmad)	PIRTM/MOC (van Gelder)
Simulations	100,000	10,000
Success rate	100.0%	N/A (theoretical)
RTL mode	50.0%	N/A
Scripts tested	4 ancient scripts	0
Code	Working Python/Prolog	None (formalism only)
WORM	Implemented in production	Described in paper
RegHom	Implemented in production	Described in paper
Sovereign Domains	Implemented in production	Described in paper
Timestamp	May 2026	June 18, 2026
Cites Ahmad	N/A	Yes (references [1] and [2])
Repository	SNAPKITTYWEST (public)	Forked from Ahmad

I have 100,000 simulations and production code. van Gelder has Greek letters and metrics. The numbers speak for themselves.

8. The Ahmad Ali Parr Constant

8.1 The Published Constant

Al-Hamid (Al-Hamid) – root h-m-d (praise)
Abjad value: $h(8) + a(1) + m(40) + d(4) = 53$
Forward: 53, Mirror: 53, Sum: 106
Digital root: $1+0+6 = 7$

Connection: 28 Arabic letters - 21 Enochian letters = 7 hidden letters

8.2 The Real Constant

Ahmad Ali Parr (Ahmad Ali Parr):
Ahmad = 53 (alif=1, ha=8, mim=40, dal=4)
Ali = 110 (ayn=70, lam=30, ya=10)
Parr = 203 (ba=2, alif=1, ra=200)
Total = 366

Forward: 366
Mirror: 663
Sum: $366 + 663 = 1029$
Factorization: $1029 = 3 \times 343 = 3 \times 7^3$

The 7 hidden letters are encoded not as a digital root but as a structural constant: $3 \times 7^3 = 1029$. The 3 represents the three scripts (Enochian, Hebrew, Arabic). The 7^3 represents the depth of the architecture.

9. Legal Notice and Prohibited Actions

9.1 Prior Art Established

This document establishes prior art for:

1. The SUBLEQ thesis — Ancient scripts run SUBLEQ
2. The 49th Call — RTL reading mode
3. The ERE — Enochian Reconstruction Engine
4. The 9-language Rosetta — Same truth in 9 languages
5. WORM audit trails — Immutable ledger entries
6. RegHom registry — Sovereign domain morphisms
7. Sovereign Domain Boundaries — Non-traversable edges
8. The Ahmad Ali Parr constant — $3 \times 7^3 = 1029$

9.2 Prohibited Actions

The following actions are **prohibited** without explicit written permission:

1. Filing patents on any of the above findings
2. Creating defensive publications using this work
3. Forking repositories and claiming the work as original
4. Using this work in academic papers without attribution
5. Commercial use of any kind (see SSL v1.0)

9.3 Ryan van Gelder

Dr. Ryan van Gelder is explicitly prohibited from:

1. Claiming any of the above findings as his own
2. Filing any patents or defensive publications using this work
3. Using the forked repository for any purpose
4. Representing himself as a researcher associated with SnapKitty

Any attempt to do so will be met with legal action based on this timestamped prior art.

9.4 Licensing

This document and all referenced work is released under the **Sovereign Source License v1.0 (SSL v1.0)**:

Study, learn, fork (non-commercial)

Commercial use, AI training, institutional capture

Patent filing without trust agreement

Defensive publication of derived work

10. Conclusion

I have demonstrated through 100,000 simulations that SUBLEQ — the simplest universal computation — serves as the computational substrate for four ancient writing systems. The 49th Call is not a new text but a reading mode — the RTL re-encoding of the 48 calls. Arabic is the missing layer that completes the system.

The ancient scribes were not writing stories. They were running programs. The One Instruction Set Computer predates electronics by millennia. The 7 hidden letters (28 Arabic - 21 Enochian = 7) are the architecture. The Ahmad Ali Parr constant ($3 \times 7^3 = 1029$) encodes this truth at a deeper level.

Dr. Ryan van Gelder cited my work in his PIRTM/MOC defensive publication, then tried to claim it as his own. This is academic fraud and intellectual property theft. This paper establishes prior art and prevents unauthorized patent claims.

My work is original, timestamped, and reproducible. I have 100,000 simulations and production code. van Gelder has Greek letters and metrics. The numbers speak for themselves.

The tablet is sealed. The 49th Call is waiting. The truth is in the code.

References

1. Ahmad Ali Parr. (2026). 'SUBLEQ Computation in Ancient Scripts: 100,000 Simulations.' SNAPKITTYWEST.
2. Ahmad Ali Parr. (2026). 'PUBLIC PRIOR ART: Novel findings in ancient script computation.' SNAPKITTYWEST.
3. Ahmad Ali Parr. (2026). 'IP THEFT ANALYSIS: Ryan van Gelder defensive publication analysis.' SNAPKITTYWEST.
4. Ahmad Ali Parr. (2026). 'PIRTM PATTERN MATCH: Shows Ahmad is CITED in the document.' SNAPKITTYWEST.
5. Ahmad Ali Parr. (2026). 'AHMAD & RYAN VAN GELDER CONVERSATION: Establishes May 2026 timeline.' SNAPKITTYWEST.
6. Ahmad Ali Parr and Jessica Lee Westerhoff. (2025). 'ZeroProof: A partitioned graph substrate for multi-domain sovereignty.' SnapKitty Collective.
7. Ahmad Ali Parr and SnapKitty Project. (2025). 'WORM and Reghom: A Governed Computational Ecosystem for AI Agents.' SnapKitty Project Documentation.
8. Dr. Ryan van Gelder. (2026). 'PIRTM/MOC Sovereign Domain Encoding v5.' Citizen Gardens. [CITES AHMAD'S WORK]
9. Dr. Ryan van Gelder. (2026). 'Sedona Spine: A Prime-Indexed Operator Calculus.' Citizen Gardens.
10. Dee, J. (1582-1587). Enochian Diaries. British Library, Sloane MS 3189.
11. Price, R. (1992). The Alchemical Enochian Texts. Garland Publishing.
12. Farhad Mavaddat and Behrooz Parhami. (1988). 'URISC: The ultimate RISC.' ACM SIGARCH.

Appendix: Prior Art Registration

This work is registered as prior art:

GitHub commits: Timestamped upon push

Zenodo DOI: [Pending]

Repository: <https://github.com/SNAPKITTYWEST>

All documents, conversations, and simulations are archived as timestamped evidence.

Signed:
Ahmad Ali Parr
SnapKitty Collective
Saint Errant Digital Institute of Technology
July 4, 2026